Pipeline de eventos de comercio electrónico

Estructuras de Datos y Algoritmos · UTEC · Maestría en Ciencia de Datos e Inteligencia Artificial · 2026-2

Grupo EDA de cuatro integrantes. Identificación nominal y aportes históricos por confirmar por el grupo. Repositorio: github.com/bryanOsmar07/Proyecto_EDA_UTEC.

Resumen

Se implementó un pipeline de diez etapas para procesar navegación y compras, consultar productos, detectar alta frecuencia y producir recomendaciones. Se integran FIFO, ordenación y búsqueda binaria, Heap, almacenamiento exacto, matriz CSR, caché LRU, B+Tree por páginas, KDTree, Bloom Filter, Count-Min Sketch, MinHash con LSH, IVF Flat, PageRank, Louvain y un servicio HTTP con Trie y dashboard. La ejecución real procesó 10.000.000 filas de REES46 y aceptó 9,993,787 eventos tras la política de deduplicación; construyó 121,903 productos y un grafo de 19,215 nodos y 12,607 aristas.

El tiempo integral observado fue 1,133.13 s y el pico RSS muestreado 1,064.26 MiB. Son una ejecución local y un pico muestreado, no una garantía de latencia o memoria para otras distribuciones. Los índices se evalúan sobre los productos derivados, no sobre diez millones de vectores distintos. Las comparaciones incluyen métodos exactos, error probabilístico, recall@5 y modularidad y estabilidad de comunidades.

Contexto y objetivo

Una tienda debe atender eventos de distinta urgencia y combinar consulta de catálogo con actividad de usuarios. Utilizar una sola estructura para todas las operaciones desperdicia orden, dispersión o aproximación. El objetivo es comprobar que la salida de cada etapa participa en una consulta, alerta o recomendación posterior y medir el costo de esa combinación.

Se eligió comercio electrónico por disponer de datos públicos de comportamiento REES46. Las vistas, carritos y compras reales conservan sus timestamps. Un generador adicional crea órdenes explícitas, prioridades de envío, retransmisiones y una cuenta de alta frecuencia para una demo controlada. Las señales sintéticas y las inferencias se identifican en las salidas.

Pregunta experimental

¿Qué compromisos de tiempo, memoria y calidad producen las estructuras probabilísticas y aproximadas frente a sus alternativas exactas, y qué aporta incorporar influencia y comunidades al servicio? La evaluación separa funcionamiento, aproximación geométrica y utilidad de negocio: recuperar vecinos o detectar frecuencia no prueba por sí solo relevancia comercial o fraude.

Datos y contrato de procesamiento

Se usan prefijos reproducibles del archivo 2019-Oct.csv: 1K, 10K, 100K, 1M y 10M filas. El archivo local pesa 5.668.612.855 bytes y se distribuye por separado por su tamaño. Se mantienen IDs de productos y usuarios, precios y timestamps, y se normalizan metadatos vacíos a unknown. La disponibilidad real queda desconocida; la demo muestra inventario simulado identificado y no se atribuyen a REES46 medios de pago que no contiene.

Eventos de origen	Aceptados	Productos	Nodos / aristas
1,000	1,000	534	10 / 1
10,000	9,996	4,148	96 / 6
100,000	99,983	20,621	843 / 159
1,000,000	999,498	63,322	4,549 / 1,420
10,000,000	9,993,787	121,903	19,215 / 12,607

Identidad y limpieza

El lector asigna un número de fila para trazas, pero la deduplicación global usa un digest BLAKE2b de 128 bits del contenido normalizado. Filas idénticas se consideran retransmisiones. En las primeras 100K se eliminan los 17 repetidos que antes tenían IDs distintos. Si se recibe source_event_id, ese identificador externo define la identidad. Sin él no puede distinguirse perfectamente una acción real idéntica de una retransmisión; la política debe ajustarse a la fuente.

Cada microlote valida tipos, zona horaria, IDs positivos y precio finito no negativo. Ordena por event_id y utiliza búsqueda binaria para repetidos del lote. Los eventos rechazados no actualizan índices. Los metadatos opcionales se completan con un contrato único para evitar fallos posteriores por campos ausentes.

Órdenes y ventanas

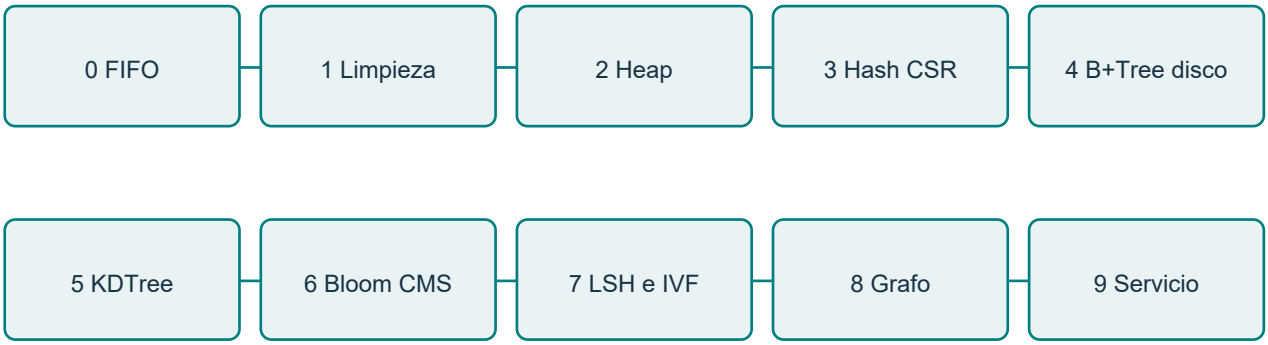
REES46 no trae order_id. Se adopta una orden inferida por user_session y hora UTC, marcada como tal. Una relación del grafo une productos distintos de una misma orden inferida; su peso cuenta órdenes compartidas. Una sesión puede dividirse o agruparse de forma imperfecta, por lo que el grafo no representa pedidos comerciales confirmados. La simulación sí produce órdenes explícitas.

La frecuencia de compras se define por usuario y hora calendario UTC. La hora se incorpora a la clave del sketch, por lo que reordenar eventos mediante el Heap no cambia su ventana. El CMS acumula durante toda la ejecución: su error depende del total de compras, aunque la consulta corresponda a una hora. El conteo exacto persiste en disco.

Alcance operacional

La ejecución es incremental y acotada por microlotes, no un stream distribuido infinito. El Heap prioriza dentro del lote. Los índices se reconstruyen en instantáneas, al final por defecto o cada N eventos si se configura snapshot-every. Las urgencias express, estándar y programada se usan cuando existen; en REES46 la compra sin etiqueta tiene la prioridad estándar por política, sin afirmar que su envío sea realmente estándar.

Arquitectura integrada y estructuras base



Esquema lógico. El filtrado protege las actualizaciones y las ramas confluyen en el servicio.

El filtrado probabilístico protege las actualizaciones exactas. El catálogo alimenta el B+Tree, las características, los conjuntos y las compras del grafo. KDTree proporciona la normalización que consume IVF. La CSR alimenta popularidad; PageRank y co-compra modifican el Top-K. El servicio consulta índices reales y expone trazas de los eventos. El diagrama completo con ramas está en docs/arquitectura.md.

Estructura	Costo relevante	Razón de selección
FIFO deque	O(1) por extremo; O(b) espacio	Evita pop(0) de listas
Ordenación / binaria	O(b log b) por lote	Limpieza ordenada y verificable
Heap	O(b) heapify; O(log b) extracción	Urgencia estable dentro del lote
Hash / LRU	O(1) esperado en RAM	Lookup de producto y localidad
CSR	O(u+z) almacenamiento	Interacciones dispersas y popularidad
B+Tree de disco	O(log_B p) páginas por punto	Rangos sin cargar todo el árbol
KDTree	Consulta favorable O(log p+k), peor O(p)	Referencia exacta en cuatro dimensiones

Notación: b tamaño de lote, p productos, u usuarios, z interacciones no nulas y B claves por página. Los costos hash se refieren a RAM; las tablas exactas de SQLite tienen sus propios índices y costos de disco. Las inserciones en nodos del B+Tree también desplazan O(B) elementos de listas. Las consultas por rango dependen del tamaño de salida.

Persistencia y memoria

SQLite conserva IDs vistos, interacciones, órdenes, alertas y conteos exactos. No se retienen diez millones de diccionarios de eventos. El B+Tree de precios es propio: páginas de 4096 bytes, separadores compuestos precio-ID, hojas enlazadas y caché LRU acotada. Reabrirlo conserva búsquedas por punto y rango sin deserializar todo el índice. Es una estructura académica sin eliminación ni recuperación transaccional ante caídas.

Catálogo, firmas, índices y grafo derivados sí crecen con sus entidades. La solución reduce retención de eventos, pero no promete memoria constante. Una instantánea reconstruye los índices y vacía la LRU de productos para evitar respuestas de versiones anteriores.

Filtrado probabilístico y estimación de frecuencia

Bloom [1] representa pertenencia mediante m bits y h hashes. Insertar enciende h bits; una consulta negativa indica ausencia y una positiva solo posibilidad. Aproximadamente f=(1-exp(-hn/m))^h. Se elige m=ceil(-n ln f/(ln 2)^2), h cercano a (m/n)ln 2. Tiempo O(h) y espacio O(m) bits, más la sobrecarga de implementación. El código usa bytearray y doble hashing BLAKE2b reproducible.

En el flujo, un positivo Bloom se confirma contra seen en SQLite antes de descartar el evento. Un negativo evita esa lectura exacta, pero la inserción persistente sigue siendo necesaria. Un set en RAM puede resultar más rápido por menores constantes. No se atribuye al sistema híbrido el ahorro del filtro aislado, pues la estructura exacta sigue almacenándose.

Count-Min Sketch [2] mantiene d filas y w columnas de contadores uint64. Actualizar toca una celda por fila; consultar toma el mínimo. w=ceil(e/epsilon), d=ceil(ln(1/delta)). Con incrementos positivos no subestima y, bajo los supuestos del análisis, sobreestima como máximo epsilon*N con probabilidad al menos 1-delta para una consulta. Tiempo O(d) y espacio O(wd); no es garantía simultánea para todas las claves sin ajustar delta.

La configuración online usa epsilon=1e-5, delta=0,01 y umbral 10 compras por usuario y hora. El diccionario exacto confirma candidatos y emite una alerta al alcanzar el umbral. La cota se aplica al total de compras de toda la ejecución porque las horas son parte de la clave y el sketch no se vacía. No se bloquean operaciones por una estimación. Las alertas son reglas experimentales de actividad, sin etiquetas de fraude real.

N eventos	Productos	FPR Bloom	Error medio CMS	Error máximo CMS
1,000	534	1.19%	0.000	0
10,000	4,148	0.99%	0.000	0
100,000	20,621	0.82%	0.052	6
1,000,000	63,322	1.01%	2.209	63
10,000,000	121,903	1.23%	29.245	647

La tabla compara pertenencia por producto y construcción de frecuencias ponderadas a partir de los eventos aceptados. Bloom se contrasta con set y CMS con Counter. La FPR usa 10.000 claves ausentes. El protocolo registra tiempo y memoria Python actual/pico y RSS. El monitor online usa usuario-hora y parámetros diferentes: sus errores no se intercambian con esta tabla sin indicar el cambio de clave.

En la ejecución de 10M se registraron 83 alertas exactas. Hubo 324 evaluaciones candidatas CMS y 1 descartadas por conteo exacto. La cota aditiva de esa ejecución fue 1.728. Estas cifras describen el protocolo observado, no precisión de un detector de fraude.

Método en corpus de 10M	Python actual MiB	Pico Python MiB
bloom	0.140	0.142
set	4.000	6.000
cms	1.042	1.044
counter	5.179	7.647

Similitud de conjuntos con MinHash y LSH

Se seleccionó MinHash con LSH [3,4]. Cada producto se representa por tokens de marca, categoría e intervalo logarítmico de precio. Jaccard compara la intersección con la unión. Esta representación expresa atributos compartidos y complementa el vector de comportamiento. Las coincidencias en unknown pueden introducir semejanza artificial.

Cada componente de firma conserva el mínimo de un hash de tokens. Bajo permutaciones min-wise, la probabilidad de igualdad es Jaccard. La implementación usa hashes afines reproducibles como aproximación. Con s componentes, construir una firma cuesta O(s|A|) y almacenarla O(s). El pipeline usa 32 componentes en ocho bandas de cuatro filas, semilla 42.

LSH genera candidatos cuando coincide una banda. Idealmente, para similitud J la probabilidad es 1-(1-J^r)^b. Más bandas recuperan más candidatos, con mayor trabajo y memoria. Los candidatos se refinan mediante Jaccard exacto y desempate por product_id. La búsqueda no completa faltantes por fuerza bruta; devolver menos de K reduce el recall.

N eventos	Productos	Recall@5 IDs	Recall con empates	LSH / exacto (ms)
1,000	534	0.530	0.720	3.38 / 64.34
10,000	4,148	0.760	0.900	16.12 / 638.70
100,000	20,621	0.780	0.970	50.70 / 2223.53
1,000,000	63,322	0.850	0.960	326.82 / 10460.04
10,000,000	121,903	0.930	1.000	351.78 / 12217.43

Los tiempos corresponden a un conjunto fijo de hasta 20 consultas por repetición, no a una consulta individual. La referencia recorre todos los conjuntos de la misma instantánea. El recall estricto compara IDs; la medida con empates acepta candidatos que alcanzan la similitud del quinto vecino exacto. Ambas se reportan para evitar esconder pérdidas bajo una política de empate.

Construcción en corpus de 10M	Python actual MiB	Pico Python MiB
lsh	134.178	134.179
exact_sets	30.112	30.112

Límites y trade-offs

LSH no garantiza tiempo sublineal en el peor caso. Una categoría dominante puede generar buckets grandes, llegando a recorrer gran parte del catálogo. Tokens y buckets Python consumen memoria adicional a las firmas; el tamaño de las firmas no es la memoria total. Más permutaciones reducen variación de estimación, pero aumentan construcción y espacio.

La comparación empírica mide recuperación de atributos. No evalúa todavía satisfacción de usuarios ni conversión. Para ello se necesitaría una partición temporal, una métrica de relevancia y resultados de recomendaciones frente a un baseline de popularidad, evitando fuga de información entre entrenamiento y prueba.

Búsqueda vectorial aproximada con IVF Flat

Se seleccionó IVF Flat [10,11], una partición por centroides con listas invertidas. Los vectores tienen log1p de precio actual y conteos view, cart y purchase, normalizados con StandardScaler. MiniBatchKMeans [12] aprende hasta 32 centroides a partir de una muestra reproducible de hasta 50.000 vectores; después asigna todo el catálogo en bloques de 8.192.

Una consulta compara el vector con los centroides, explora nprobe=4 listas y calcula distancias euclidianas exactas dentro de los candidatos. El error proviene de omitir listas. Con nprobe=nlist se explora todo el corpus: una prueba exige coincidencia con fuerza bruta. Se conservan todos los vectores en float32; no se aplica compresión PQ.

Con c centroides y dimensión d, asignar p vectores cuesta aproximadamente O(pcd). La consulta cuesta O(cd+qd+q log q) en esta implementación, con q candidatos; el ordenamiento de selección se realiza en bloques. Si las listas están equilibradas, q se aproxima a p*nprobe/c, pero en el peor caso q=p. Espacio O(pd+p+cd), más mapas Python. El entrenamiento cuesta O(tscd) para t iteraciones y s vectores de muestra.

N eventos	Vectores	Recall@5 IDs	IVF / brute (ms)	KDTree (ms)
1,000	534	1.000	7.31 / 6.81	11.54
10,000	4,148	1.000	10.07 / 19.90	12.60
100,000	20,621	1.000	14.33 / 57.29	8.22
1,000,000	63,322	1.000	65.93 / 280.63	11.51
10,000,000	121,903	1.000	157.99 / 535.40	16.59

El conjunto exacto de referencia usa el mismo float32 y un desempate por ID. KDTree emplea float64 y se verifica con tolerancia. Los tiempos cubren hasta 20 consultas fijas por repetición, instrumentadas de igual forma. Recall 1 en esa muestra no garantiza exactitud general ni sustituye un estudio de nprobe y distribuciones de mayor dimensión.

Construcción en corpus de 10M	Python actual MiB	Pico Python MiB
ivf	15.962	18.584
kd_tree	1.310	1.310

Selección técnica

IVF permite explicar con claridad la partición, la búsqueda y la relación entre recall y listas exploradas. Se aprovecha una dependencia ya presente y se mantiene la implementación de listas y consulta propia. HNSW sería otra alternativa válida con decisiones de conectividad y búsqueda adicionales; no se sostiene que IVF sea superior en todo escenario. En cuatro dimensiones, KDTree puede ser competitivo o más rápido.

Los centroides e índices se reconstruyen por instantánea, de modo que catálogo y normalización sean consistentes. El precio actual reemplaza la mediana histórica para limitar el estado de características; el pipeline histórico conserva medianas para comparación. Esta decisión reduce retención pero cambia la representación y se documenta expresamente.

Influencia y comunidades en el grafo del dominio

PageRank [5] itera una caminata ponderada con teleportación alpha=0,85, tol=1e-8 y máximo 500 iteraciones. Reparte masa de nodos sin salida y produce una distribución normalizada. El costo es O(t(V+E)) para t iteraciones, con espacio O(V+E). Se compara con grado ponderado, O(V+E). En grafos no dirigidos ambos rankings pueden estar relacionados; se registran sus Top-10 sin forzar una superioridad comercial.

Louvain [6] mueve nodos para aumentar modularidad y agrega comunidades en supernodos. Es heurístico, depende de resolución y no garantiza óptimo global. La alternativa asíncrona de propagación de etiquetas [7] usa etiquetas dominantes entre vecinos. Cada barrido recorre nodos y aristas, pero las iteraciones y convergencia dependen del grafo. Ambas requieren O(V+E) para la representación y estado auxiliar.

N eventos	Algoritmo	Modularidad	Comunidades	ARI estabilidad
100,000	louvain	0.910	739	1.000
100,000	label_propagation	0.875	745	0.918
1,000,000	louvain	0.787	3664	0.840
1,000,000	label_propagation	0.717	3725	0.493
10,000,000	louvain	0.629	13722	0.697
10,000,000	label_propagation	0.445	14324	0.696

La tabla usa tres corridas por algoritmo y grafo, con semillas 42, 43 y 44. ARI se promedia sobre parejas y es invariante a renombrar comunidades. Las listas completas de tamaños, tiempos y desviaciones están en pipeline_summary.json y los CSV de benchmarks. Para grafos sin aristas se usa modularidad 0 por convención y comunidades singleton.

Método en corpus de 10M	Python actual MiB	Pico Python MiB
pagerank	1.130	5.209
degree	0.563	0.845
louvain	3.140	28.395
label_propagation	3.307	5.408

Integración y significado

Una compra aceptada actualiza una membresía de orden-producto. Al construir la instantánea se enumeran pares de la misma orden y se suman pesos. Los vecinos de co-compra generan recomendaciones complementarias, mientras PageRank contribuye al score final. La comunidad y la influencia de cada candidato aparecen en el panel, junto con las razones de recomendación.

Una modularidad alta no demuestra comunidades verdaderas ni utilidad comercial. El proxy de órdenes por sesión-hora puede introducir o perder aristas. Asimismo, estabilidad alta puede reflejar componentes aislados simples. Por eso se reportan también cantidades y tamaños, y se mantiene una demo con órdenes sintéticas explícitas para explicar el mecanismo sin presentar inferencias como hechos confirmados.

Diseño experimental y reproducibilidad

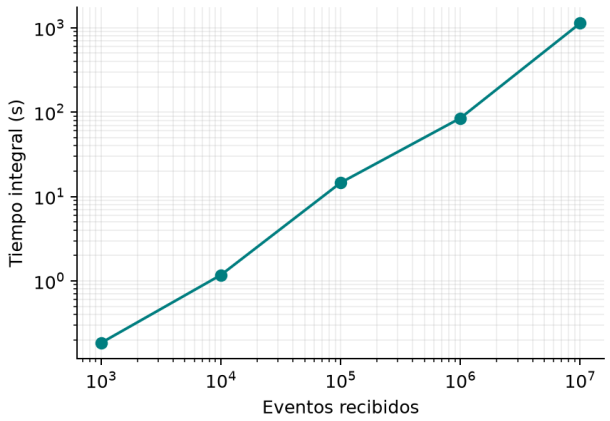
La unidad de escala es el evento de origen. Los índices y el grafo se construyen sobre todas las entidades derivadas de cada prefijo. Se informan por separado eventos aceptados, productos, usuarios, nodos y aristas. Recorrer 10M filas no prueba un índice de 10M productos distintos: el catálogo real contiene menos productos y determina la carga de las consultas.

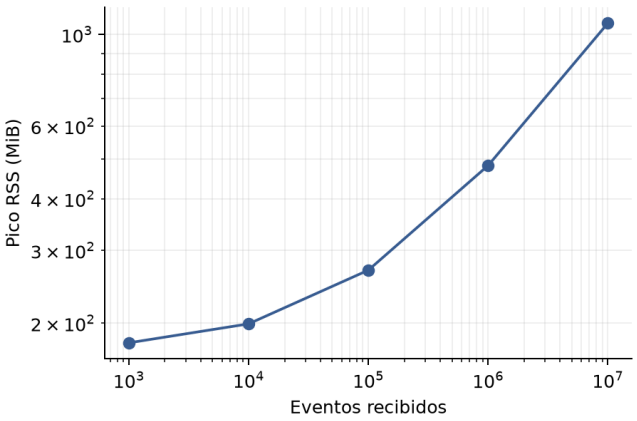
Aspecto	Protocolo
Escalas	1K, 10K, 100K, 1M y 10M filas REES46
Repeticiones	100 hasta 100K; repeticiones observadas por escala en CSV; 10M una ejecución
Consultas	Hasta 20 IDs fijos con semilla 42; K=5; alternancia de métodos
Plataforma	Python 3.12.10; Windows 11; 15.67 GiB RAM; un hilo nativo
Tiempo	perf_counter; incluye instrumentación tracemalloc en comparaciones
Memoria	Python actual/pico, RSS actual y pico muestreado cada 5 ms; integral a 20 ms
Referencias	Set/Counter, Jaccard exhaustivo, KNN exacto, grado y propagación de etiquetas

La desviación estándar es muestral y queda no estimable si hay una sola observación. Las repeticiones grandes se reducen por tiempo de procesamiento, I/O y RAM compartida de un equipo de 16 GB. No se inventa desviación cero para una ejecución. Las mediciones se hicieron en una estación de trabajo compartida y no aislada; la planificación y cachés introducen ruido. Por ello sirven para evaluar esta implementación, sin prometer comparaciones universales.

La instrumentación de consultas mide buffers y estado temporal de consulta sobre índices ya construidos. La memoria de construcción se mide en un experimento separado y se entrega en index_build_raw.csv e index_build_summary.csv. Los tamaños de payload se distinguen de memoria total. El RSS base del proceso incluye bibliotecas y corpus, por lo que no representa memoria exclusiva de un algoritmo.

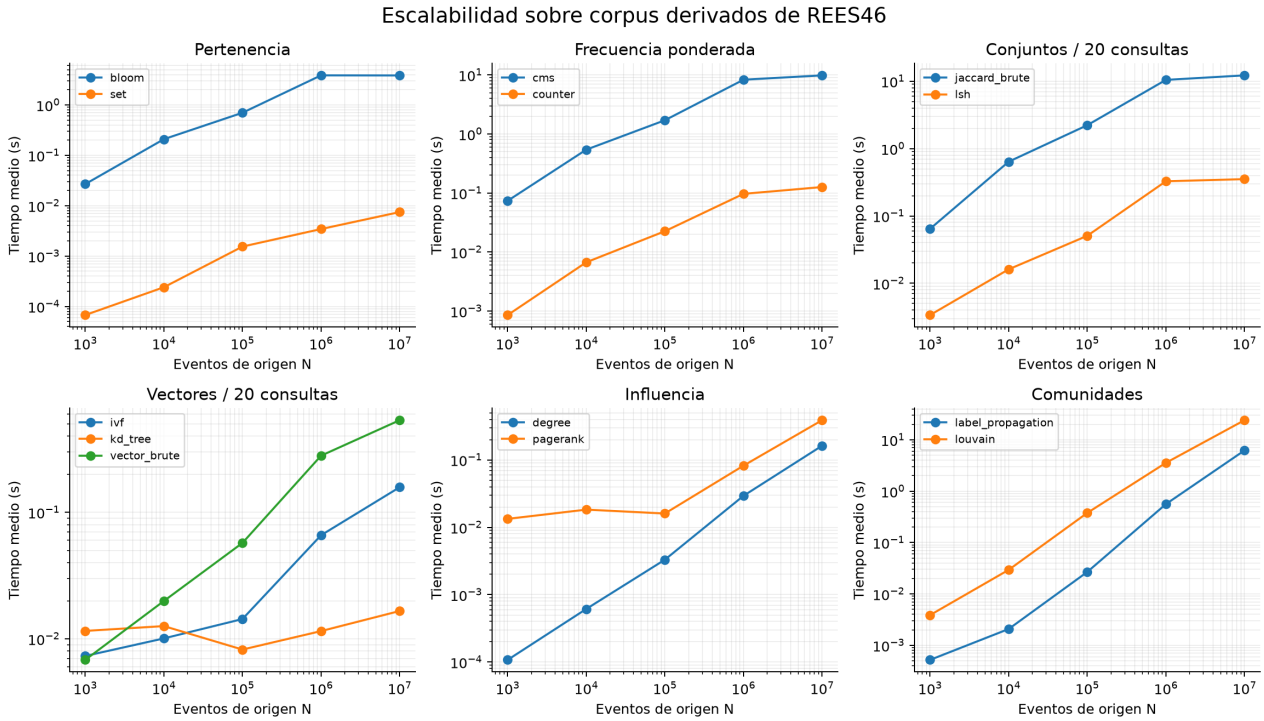
Se conserva cada repetición en CSV con tamaño real del corpus, semilla, método, tiempo y memoria. Las figuras se generan leyendo esos archivos; no contienen tiempos escritos manualmente. Los experimentos históricos permanecen separados y no se mezclan sus protocolos con los nuevos. El README proporciona instalación, comandos y límites; el workflow de CI ejecuta pruebas sin necesitar el dataset.





Resultados integrales y discusión

Eventos recibidos	Tiempo integral s	Pico RSS MiB	Aceptados
1,000	0.18	179.19	1,000
10,000	1.18	199.17	9,996
100,000	14.60	268.65	99,983
1,000,000	85.09	481.36	999,498
10,000,000	1,133.13	1,064.26	9,993,787



Los gráficos log-log contrastan el crecimiento observado de estas operaciones con sus mecanismos. Bloom y CMS crecen con las claves insertadas en el benchmark ponderado; la búsqueda exacta recorre el catálogo; LSH e IVF dependen de la distribución de buckets y listas; PageRank y comunidades dependen de nodos y aristas. La cardinalidad derivada no crece necesariamente de forma lineal con eventos de origen, por lo que una pendiente frente a N no estima por sí sola el exponente teórico del algoritmo.

El Heap y el B+Tree aportan prioridades y rangos; su propósito no es acelerar cada etapa indiscriminadamente. El filtro probabilístico puede gastar más CPU que un set en RAM. En ANN, aumentar candidatos mejora calidad a costa de tiempo. En comunidades, una alternativa más rápida puede ofrecer menor modularidad o menor estabilidad. Los CSV conservan resultados favorables y desfavorables para que estas decisiones puedan sustentarse.

Conclusiones y límites

La implementación conecta las diez etapas y produce consultas de rango, autocompletado, recomendaciones Top-K, alertas confirmadas e información de influencia/comunidades. Las pruebas comprueban persistencia por páginas, equivalencia exacta al explorar todas las listas IVF, normalización, duplicados, entrada vacía y consumo de resultados por el servicio. La ejecución de 10M demuestra viabilidad local bajo la distribución y configuración medidas.

Persisten límites explícitos: órdenes inferidas en REES46, ausencia de verdad de fraude, score heurístico sin validación comercial, prioridad por microlote, reconstrucción de índices por instantánea y estado derivado creciente. Las repeticiones de gran escala son reducidas. El B+Tree es académico y no implementa concurrencia, eliminación o recuperación ante caídas. Estos límites acotan las conclusiones y orientan extensiones, sin convertir resultados de muestra en garantías generales.

La distribución propuesta entre cuatro integrantes está en el README y el guion de sustentación. Los nombres y aportes pasados deben confirmarse por el grupo; no se atribuye autoría histórica ficticia. La presentación y demo se acompañan de preguntas sobre cualquier etapa. La asistencia y defensa oral son actividades individuales que el software no puede sustituir.

Referencias

[1] Bloom (1970). Space/time trade-offs in hash coding with allowable errors. CACM 13, 422-426. doi:10.1145/362686.362692.

[2] Cormode y Muthukrishnan (2005). An improved data stream summary. Journal of Algorithms 55, 58-75. doi:10.1016/j.jalgor.2003.12.001.

[3] Broder (1997). On the resemblance and containment of documents. SEQUENCES, 21-29. doi:10.1109/SEQUEN.1997.666900.

[4] Indyk y Motwani (1998). Approximate nearest neighbors. STOC, 604-613. doi:10.1145/276698.276876.

[5] Brin y Page (1998). The anatomy of a large-scale hypertextual Web search engine. doi:10.1016/S0169-7552(98)00110-X.

[6] Blondel et al. (2008). Fast unfolding of communities in large networks. doi:10.1088/1742-5468/2008/10/P10008.

[7] Raghavan et al. (2007). Near linear time algorithm to detect community structures. doi:10.1103/PhysRevE.76.036106.

[8] Leskovec, Rajaraman y Ullman (2020). Mining of Massive Datasets, tercera edición. mmds.org.

[9] Cormode. Count-Min Sketch. dimacs.rutgers.edu/~graham/pubs/papers/encalgs-cm.pdf.

[10] Sivic y Zisserman (2003). Video Google. robots.ox.ac.uk/~vgg/publications/2003/Sivic03/sivic03.pdf.

[11] Faiss. Faiss indexes. github.com/facebookresearch/faiss/wiki/Faiss-indexes.

[12] scikit-learn 1.9.1. MiniBatchKMeans y KDTree. scikit-learn.org/stable/.

[13] NetworkX 3.7. PageRank. networkx.org/documentation/stable/.

[14] NetworkX 3.7. Louvain y propagación de etiquetas. networkx.org/documentation/stable/reference/algorithms/community.html.